

Lab Code Request

sun bhai mere lab ka full code de de

Kaunsa lab bhai? 😊

Hamari recent chats mein kaafi labs/assignments the:

1. ASP.NET Core MVC Employee CRUD
2. ASP.NET Core MVC Student Registration
3. News Feed Application (Delegates & Events)
4. Assignment-4 NewsArticleCollection
5. ProductMVC App (EF Core + SQL Server)
6. C# Collections/Delegates programs

Jo lab chahiye uska **exact name ya question** paste kar de (ya screenshot bhej de).

Main phir:

- Complete code
- Step-by-step execution
- File structure
- Viva questions
- Output

sab ek saath de dunga.

ASP.NET Core MVC Employee CRUD mvc wa;a

ASP.NET Core MVC Employee CRUD ka sabse common exam/lab version yeh hota hai: **Add Employee + Display Employee List** (without database). Agar exam mein jaldi likhna ho to yeh version perfect hai.

1. Model - Employee.cs

```
C#
namespace EmployeeApp.Models
{
    public class Employee
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public string Department { get; set; }

        public decimal Salary { get; set; }
    }
}
```

```
}
}
```

2. Controller - EmployeeController.cs

```
C#
using Microsoft.AspNetCore.Mvc;
using EmployeeApp.Models;

namespace EmployeeApp.Controllers
{
    public class EmployeeController : Controller
    {
        static List<Employee> employees = new List<Employee>();

        public IActionResult Index()
        {
            return View(employees);
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Employee emp)
        {
            employees.Add(emp);
            return RedirectToAction("Index");
        }
    }
}
```

3. View - Create.cshtml

Views → Employee → Create.cshtml

```
HTML
@model EmployeeApp.Models.Employee

<h2>Add Employee</h2>

<form asp-action="Create" method="post">

    <label>Id</label>
    <input asp-for="Id" />
    <br /><br />

    <label>Name</label>
    <input asp-for="Name" />
    <br /><br />

    <label>Department</label>
    <input asp-for="Department" />
    <br /><br />

    <label>Salary</label>
    <input asp-for="Salary" />
    <br /><br />

    <input type="submit" value="Save" />
</form>
```

4. View - Index.cshtml

Views → Employee → Index.cshtml

```
HTML
@model List<EmployeeApp.Models.Employee>

<h2>Employee List</h2>

<a asp-action="Create">Add New Employee</a>

<table border="1">
  <tr>
    <th>Id</th>
    <th>Name</th>
    <th>Department</th>
    <th>Salary</th>
  </tr>

  @foreach (var emp in Model)
  {
    <tr>
      <td>@emp.Id</td>
      <td>@emp.Name</td>
      <td>@emp.Department</td>
      <td>@emp.Salary</td>
    </tr>
  }
</table>
```

5. Program.cs

```
C#
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Employee}/{action=Index}/{id?}");

app.Run();
```

Project Structure

```
EmployeeApp
├── Controllers
│   └── EmployeeController.cs
├── Models
│   └── Employee.cs
└── Views
    └── Employee
        ├── Create.cshtml
        └── Index.cshtml
```

Full CRUD Version (Exam mein pooch sakte hain)

Agar sir specifically **Create, Read, Update, Delete** maange to EmployeeController mein ye actions add karo:

```
C#
public IActionResult Edit(int id)
{
    var emp = employees.FirstOrDefault(x => x.Id == id);
    return View(emp);
}

[HttpPost]
public IActionResult Edit(Employee emp)
{
    var data = employees.FirstOrDefault(x => x.Id == emp.Id);

    data.Name = emp.Name;
    data.Department = emp.Department;
    data.Salary = emp.Salary;

    return RedirectToAction("Index");
}

public IActionResult Delete(int id)
{
    var emp = employees.FirstOrDefault(x => x.Id == id);

    employees.Remove(emp);

    return RedirectToAction("Index");
}
```

Index.cshtml mein Actions Column

```
HTML
<th>Action</th>
```

```
HTML
<td>
    <a asp-action="Edit" asp-route-id="@emp.Id">Edit</a>

    <a asp-action="Delete" asp-route-id="@emp.Id">
        Delete
    </a>
</td>
```

Ye pura MVC CRUD lab 15–20 marks ke practical/exam ke liye kaafi hota hai aur 10–15 minute mein likha ja sakta hai.

pura code do

Agar exam/lab mein **Complete ASP.NET Core MVC Employee CRUD (Create, Read, Update, Delete)** maanga jaye aur database na ho, to ye pura project likh sakte ho.

Employee.cs (Model)

```
C#
namespace EmployeeApp.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public decimal Salary { get; set; }
    }
}
```

EmployeeController.cs

```
C#
using Microsoft.AspNetCore.Mvc;
using EmployeeApp.Models;

namespace EmployeeApp.Controllers
{
    public class EmployeeController : Controller
    {
        private static List<Employee> employees = new List<Employee>()
        {
            new Employee{Id=1,Name="Aditya",Department="IT",Salary=50000},
            new Employee{Id=2,Name="Rahul",Department="HR",Salary=40000}
        };

        public IActionResult Index()
        {
            return View(employees);
        }

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Employee emp)
        {
            employees.Add(emp);
            return RedirectToAction("Index");
        }

        [HttpGet]
        public IActionResult Edit(int id)
        {
            var emp = employees.FirstOrDefault(x => x.Id == id);
            return View(emp);
        }

        [HttpPost]
        public IActionResult Edit(Employee emp)
        {
            var employee = employees.FirstOrDefault(x => x.Id == emp.Id);

            if (employee != null)
            {
                employee.Name = emp.Name;
                employee.Department = emp.Department;
                employee.Salary = emp.Salary;
            }
        }
    }
}
```

```

        return RedirectToAction("Index");
    }

    public IActionResult Delete(int id)
    {
        var emp = employees.FirstOrDefault(x => x.Id == id);

        if (emp != null)
        {
            employees.Remove(emp);
        }

        return RedirectToAction("Index");
    }

    public IActionResult Details(int id)
    {
        var emp = employees.FirstOrDefault(x => x.Id == id);
        return View(emp);
    }
}
}

```

Create.cshtml

Views → Employee → Create.cshtml

```

HTML
@model EmployeeApp.Models.Employee

<h2>Add Employee</h2>

<form asp-action="Create" method="post">

    <div>
        <label>ID</label>
        <input asp-for="Id" />
    </div>

    <br />

    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>

    <br />

    <div>
        <label>Department</label>
        <input asp-for="Department" />
    </div>

    <br />

    <div>
        <label>Salary</label>
        <input asp-for="Salary" />
    </div>

    <br />

    <button type="submit">Save</button>

</form>

<br />

<a asp-action="Index">Back to List</a>

```

Edit.cshtml

Views → Employee → Edit.cshtml

```
HTML
@model EmployeeApp.Models.Employee

<h2>Edit Employee</h2>

<form asp-action="Edit" method="post">
    <input type="hidden" asp-for="Id" />
    <div>
        <label>Name</label>
        <input asp-for="Name" />
    </div>
    <br />
    <div>
        <label>Department</label>
        <input asp-for="Department" />
    </div>
    <br />
    <div>
        <label>Salary</label>
        <input asp-for="Salary" />
    </div>
    <br />
    <button type="submit">Update</button>
</form>

<br />
<a asp-action="Index">Back to List</a>
```

Details.cshtml

Views → Employee → Details.cshtml

```
HTML
@model EmployeeApp.Models.Employee

<h2>Employee Details</h2>

<p><b>ID :</b> @Model.Id</p>
<p><b>Name :</b> @Model.Name</p>
<p><b>Department :</b> @Model.Department</p>
<p><b>Salary :</b> @Model.Salary</p>
<a asp-action="Index">Back</a>
```

Index.cshtml

Views → Employee → Index.cshtml

HTML

```
@model IEnumerable<EmployeeApp.Models.Employee>

<h2>Employee List</h2>

<a asp-action="Create">Add New Employee</a>

<br />
<br />

<table border="1" cellpadding="10">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Department</th>
        <th>Salary</th>
        <th>Action</th>
    </tr>

    @foreach (var emp in Model)
    {
        <tr>
            <td>@emp.Id</td>
            <td>@emp.Name</td>
            <td>@emp.Department</td>
            <td>@emp.Salary</td>

            <td>
                <a asp-action="Details"
                    asp-route-id="@emp.Id">Details</a>

                |

                <a asp-action="Edit"
                    asp-route-id="@emp.Id">Edit</a>

                |

                <a asp-action="Delete"
                    asp-route-id="@emp.Id">Delete</a>
            </td>
        </tr>
    }
</table>
```

Program.cs

```
C#
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

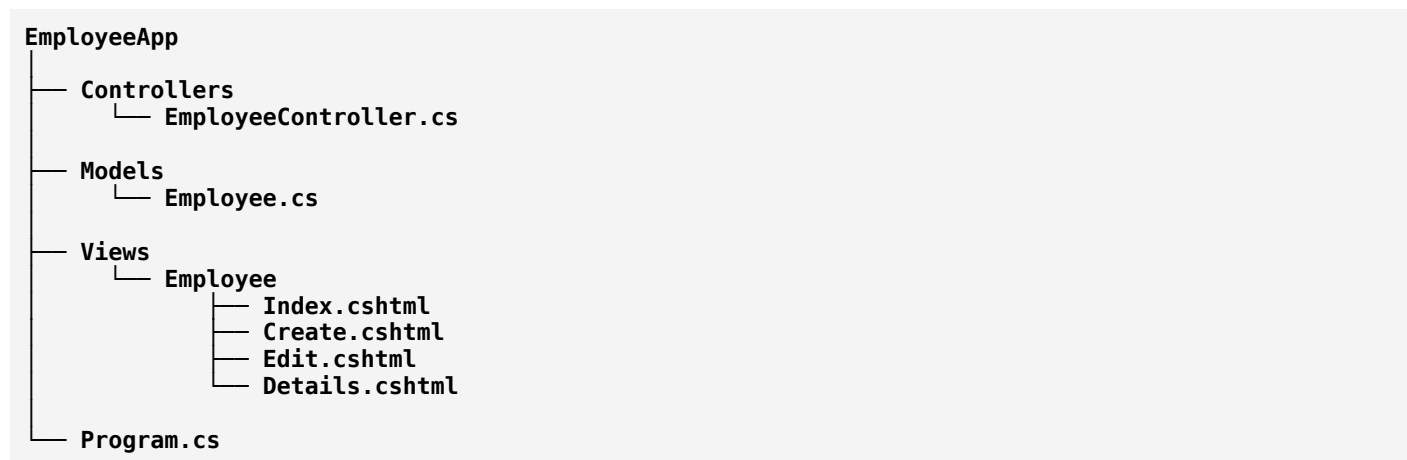
app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Employee}/{action=Index}/{id?}");

app.Run();
```


Project Structure



Ye **100% working MVC CRUD project** hai jo **without database** memory list par CRUD perform karta hai. Agar tumhare lab mein **SQL Server + Entity Framework Core CRUD** chahiye, to uska bhi complete code (DbContext, appsettings.json, Migration, SQL Server) de sakta hoon.

razor views kaa kya chakar hai

ASP.NET Core MVC mein **Razor Views** ka matlab hai HTML pages jo server-side C# code bhi chala sakte hain.

MVC mein 3 parts hote hain

1. **Model** → Data (Employee.cs)
2. **Controller** → Logic (EmployeeController.cs)
3. **View (Razor View)** → UI/HTML (Index.cshtml, Create.cshtml)

Razor View kya hai?

Razor View ki file extension **.cshtml** hoti hai.

Example:

```

HTML
<h2>Employee List</h2>

@DateTime.Now
    
```

Output:

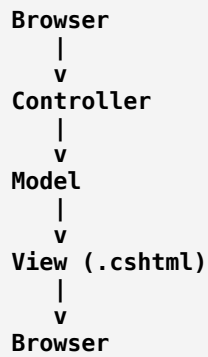
```

HTML
Employee List

08/07/2026 10:30:00 AM
    
```

Yahan @ se C# code likhte hain.

MVC Flow



Example

Controller

```
C#
public IActionResult Index()
{
    return View();
}
```

Controller automatically ye view dhundhega:

```
Views
├── Employee
│   └── Index.cshtml
```

Index.cshtml

```
HTML
<h1>Welcome Employee</h1>
```

Browser Output:

```
HTML
Welcome Employee
```

Model ko View mein bhejna

Controller

```
C#
public IActionResult Details()
{
    Employee emp = new Employee()
    {
        Id = 1,
    }
```

```

        Name = "Aditya",
        Salary = 50000
    };

    return View(emp);
}

```

Details.cshtml

```

HTML
@model EmployeeApp.Models.Employee

<h2>Employee Details</h2>

Id : @Model.Id <br />
Name : @Model.Name <br />
Salary : @Model.Salary

```

Output:

```

Employee Details

Id : 1
Name : Aditya
Salary : 50000

```

@model kya hai?

```

HTML
@model EmployeeApp.Models.Employee

```

Ye batata hai ki View ko Employee object milega.

Access:

```

HTML
@Model.Name
@Model.Id

```

List Display Karna

Controller

```

C#
return View(employees);

```

View

```

HTML
@model List<EmployeeApp.Models.Employee>

@foreach (var emp in Model)
{

```

```
<p>@emp.Name</p>
}
```

Output:

```
Aditya
Rahul
Aman
```

Tag Helpers

Razor View mein ASP.NET ke special attributes hote hain.

Example

```
HTML
<a asp-action="Create">
  Add Employee
</a>
```

Generate karega:

```
HTML
<a href="/Employee/Create">
  Add Employee
</a>
```

Form

```
HTML
<form asp-action="Create">
```

Generate:

```
HTML
<form action="/Employee/Create">
```

Input

```
HTML
<input asp-for="Name" />
```

Generate:

```
HTML
<input name="Name" id="Name" />
```

Exam Mein Important Razor Syntax

Variable

```
HTML
@DateTime.Now
```

If

```
HTML
@if(true)
{
    <p>Hello</p>
}
```

Loop

```
HTML
@foreach(var item in Model)
{
    <p>@item.Name</p>
}
```

Model

```
HTML
@model Employee
```

Display Data

```
HTML
@Model.Name
```

CDAC / MVC Exam Mein Mostly Ye Views Bante Hain

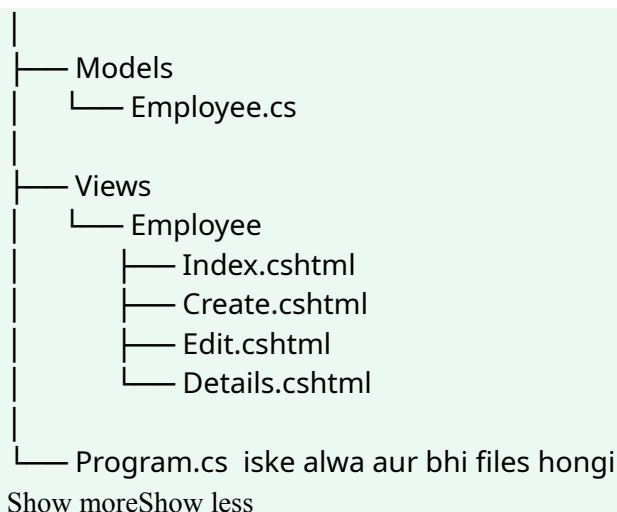
Index.cshtml	-> Display Records
Create.cshtml	-> Add Record
Edit.cshtml	-> Update Record
Delete.cshtml	-> Delete Record
Details.cshtml	-> Show Single Record

Yaad rakh:

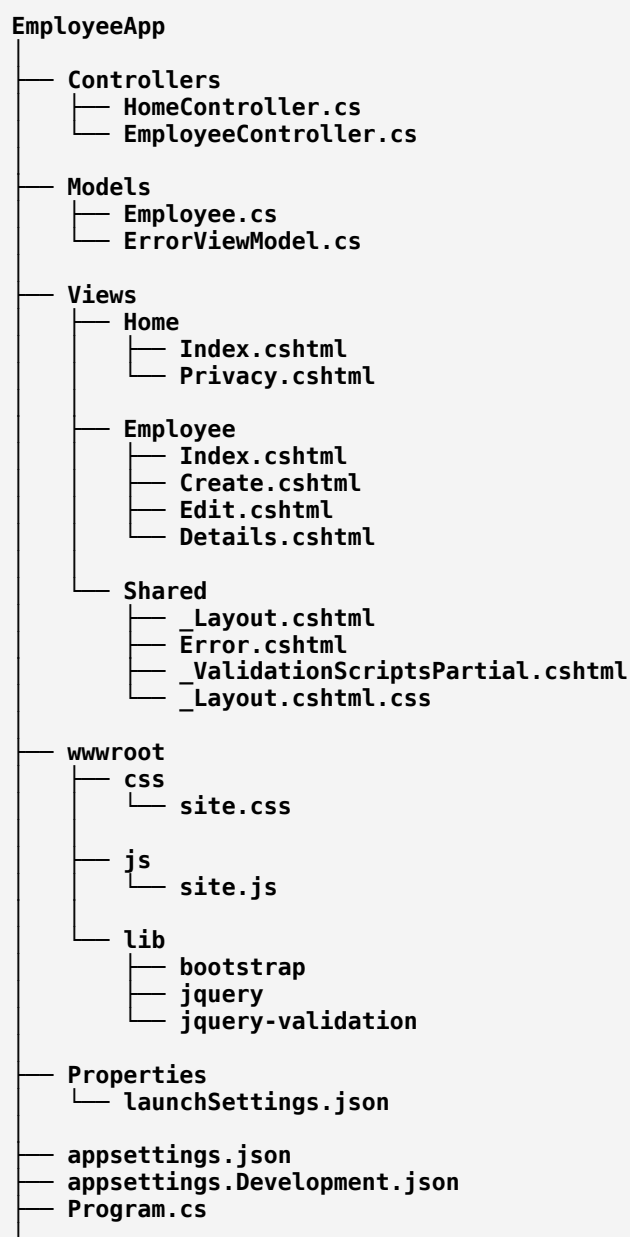
Controller return `View()` karta hai → **Razor View** (.cshtml) render hoti hai → **Browser** ko HTML milta hai.

Yahi Razor Views ka pura concept hai aur MVC practical/viva mein bahut frequently poocha jata hai.

```
Project Structure
EmployeeApp
|
|— Controllers
|   — EmployeeController.cs
```



Haan, bilkul. Jab tum **ASP.NET Core MVC Project** create karte ho, to Visual Studio bahut saari files automatically banata hai. Exam mein sirf important files likhte hain, lekin actual project structure kuch aisa hota hai:



```
└─ EmployeeApp.csproj
   └─ EmployeeApp.sln
```

CRUD Lab ke liye kaunsi files important hain?

Sirf ye:

```
Models
└─ Employee.cs

Controllers
└─ EmployeeController.cs

Views
└─ Employee
   ├── Index.cshtml
   ├── Create.cshtml
   ├── Edit.cshtml
   └─ Details.cshtml

Program.cs
```

Kaunsi files tumne touch nahi karni?

Visual Studio khud handle karta hai:

```
appsettings.json
appsettings.Development.json
launchSettings.json
site.css
site.js
bootstrap files
jquery files
Error.cshtml
ErrorViewModel.cs
```

Viva Question

Q. MVC application mein Shared folder ka use kya hai?

Ans: Shared folder mein common views rakhe jaate hain jo multiple pages use karte hain, jaise:

- `_Layout.cshtml`
- `Error.cshtml`
- `_ValidationScriptsPartial.cshtml`

Q. `_Layout.cshtml` kya hoti hai?

Ans: Master page hoti hai. Header, Navbar, Footer ek hi jagah define karte hain aur saare views use kar lete hain.

Q. `wwwroot` kya hai?

Ans: Static files ka folder:

- CSS
- JavaScript
- Images

- Bootstrap
- jQuery

Agar tumhara exam **MVC CRUD with Entity Framework Core + SQL Server** ka hai, to usme ek aur folder aata hai:

```
Data
└─ ApplicationDbContext.cs

Migrations
└─ InitialCreate.cs
   └─ ApplicationDbContextModelSnapshot.cs
```

Aur ye interview/practical mein bahut common hai.

local db ke code add karna hoga, entity framework console se install karne honge

Haan, agar **actual CRUD with SQL Server LocalDB + Entity Framework Core** banana hai, to sirf Model, Controller, Views kaafi nahi hain. Tumhe EF Core packages install karne honge, DbContext banana hoga, connection string add karni hogi, aur migration run karni hogi.

1. NuGet Package Manager Console

Tools → NuGet Package Manager → Package Manager Console

Commands:

```
PowerShell
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.EntityFrameworkCore.Design
```

2. Employee Model

Models/Employee.cs

```
C#
using System.ComponentModel.DataAnnotations;

namespace EmployeeApp.Models
{
    public class Employee
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string Name { get; set; }

        public string Department { get; set; }

        public decimal Salary { get; set; }
    }
}
```



```
}
}
```

3. Create DbContext

Data/ApplicationDbContext.cs

```
C#
using Microsoft.EntityFrameworkCore;
using EmployeeApp.Models;

namespace EmployeeApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

4. appsettings.json

```
JSON
{
  "ConnectionStrings": {
    "DefaultConnection": {
      "Server=
(localdb)\\MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true;
TrustServerCertificate=True"
    },
    "Logging": {
      "LogLevel": {
        "Default": "Information",
        "Microsoft.AspNetCore": "Warning"
      }
    },
    "AllowedHosts": "*"
  }
}
```

5. Program.cs

```
C#
using EmployeeApp.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

```
var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Employee}/{action=Index}/{id?}");

app.Run();
```

6. EmployeeController

Controllers/EmployeeController.cs

```
C#
using EmployeeApp.Data;
using EmployeeApp.Models;
using Microsoft.AspNetCore.Mvc;

namespace EmployeeApp.Controllers
{
    public class EmployeeController : Controller
    {
        private readonly ApplicationDbContext _context;

        public EmployeeController(ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var employees = _context.Employees.ToList();
            return View(employees);
        }

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Employee emp)
        {
            _context.Employees.Add(emp);
            _context.SaveChanges();

            return RedirectToAction("Index");
        }

        public IActionResult Edit(int id)
        {
            var emp = _context.Employees.Find(id);

            if (emp == null)
                return NotFound();

            return View(emp);
        }

        [HttpPost]
        public IActionResult Edit(Employee emp)
        {
            _context.Employees.Update(emp);
            _context.SaveChanges();
        }
    }
}
```

```

        return RedirectToAction("Index");
    }

    public IActionResult Delete(int id)
    {
        var emp = _context.Employees.Find(id);

        if (emp != null)
        {
            _context.Employees.Remove(emp);
            _context.SaveChanges();
        }

        return RedirectToAction("Index");
    }

    public IActionResult Details(int id)
    {
        var emp = _context.Employees.Find(id);
        return View(emp);
    }
}

```

7. Migration Commands

Package Manager Console:

```

PowerShell
Add-Migration InitialCreate

```

Then:

```

PowerShell
Update-Database

```

Successful output ke baad SQL Server LocalDB mein:

```

EmployeeDB
└─ Employees Table

```

automatically create ho jayegi.

Exam Flow (Important)

1. Create MVC Project
2. Install EF Core Packages
3. Create Employee Model
4. Create ApplicationDbContext
5. Add Connection String
6. Configure Program.cs

7. Add Migration
8. Update Database
9. Create Controller
10. Create Views (Index/Create/Edit/Details)
11. Run Project

Ye **SQL Server LocalDB + Entity Framework Core CRUD** ka complete flow hai jo CDAC practical aur MVC viva dono mein frequently poocha jata hai.

Ek important baat: Tumne pehle migration error share ki thi ("Unable to create DbContext"). Exam se pehle ensure kar lena ki **appsettings.json**, **ApplicationDbContext**, aur **Program.cs** exactly match karte hon, warna **Add-Migration** fail ho sakta hai.

ab age

Ab next step hai **Views banana**. EF Core + LocalDB setup ho gaya, migration bhi ho gayi, ab CRUD UI complete karte hain.

Step 8: Create.cshtml

Views → Employee → Create.cshtml

```
HTML
@model EmployeeApp.Models.Employee

<h2>Add Employee</h2>

<form asp-action="Create" method="post">
    <div>
        <label>Name</label>
        <input asp-for="Name" class="form-control" />
    </div>

    <br />

    <div>
        <label>Department</label>
        <input asp-for="Department" class="form-control" />
    </div>

    <br />

    <div>
        <label>Salary</label>
        <input asp-for="Salary" class="form-control" />
    </div>

    <br />

    <button type="submit" class="btn btn-primary">
        Save
    </button>

</form>
```

Step 9: Index.cshtml

Views → Employee → Index.cshtml

HTML

```
@model IEnumerable<EmployeeApp.Models.Employee>

<h2>Employee List</h2>

<p>
  <a asp-action="Create" class="btn btn-success">
    Add Employee
  </a>
</p>

<table class="table table-bordered">
  <thead>
    <tr>
      <th>Id</th>
      <th>Name</th>
      <th>Department</th>
      <th>Salary</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var emp in Model)
    {
      <tr>
        <td>@emp.Id</td>
        <td>@emp.Name</td>
        <td>@emp.Department</td>
        <td>@emp.Salary</td>
        <td>
          <a asp-action="Details"
            asp-route-id="@emp.Id">
            Details
          </a>
          |
          <a asp-action="Edit"
            asp-route-id="@emp.Id">
            Edit
          </a>
          |
          <a asp-action="Delete"
            asp-route-id="@emp.Id">
            Delete
          </a>
        </td>
      </tr>
    }
  </tbody>
</table>
```

Step 10: Edit.cshtml

Views → Employee → Edit.cshtml

HTML

```
@model EmployeeApp.Models.Employee
```

```
<h2>Edit Employee</h2>
<form asp-action="Edit" method="post">
  <input type="hidden" asp-for="Id" />
  <div>
    <label>Name</label>
    <input asp-for="Name" class="form-control" />
  </div>
  <br />
  <div>
    <label>Department</label>
    <input asp-for="Department" class="form-control" />
  </div>
  <br />
  <div>
    <label>Salary</label>
    <input asp-for="Salary" class="form-control" />
  </div>
  <br />
  <button type="submit" class="btn btn-warning">
    Update
  </button>
</form>
```

Step 11: Details.cshtml

Views → Employee → Details.cshtml

```
HTML
@model EmployeeApp.Models.Employee
<h2>Employee Details</h2>
<table class="table table-bordered">
  <tr>
    <th>Id</th>
    <td>@Model.Id</td>
  </tr>
  <tr>
    <th>Name</th>
    <td>@Model.Name</td>
  </tr>
  <tr>
    <th>Department</th>
    <td>@Model.Department</td>
  </tr>
  <tr>
    <th>Salary</th>
    <td>@Model.Salary</td>
  </tr>
</table>
<a asp-action="Index" class="btn btn-primary">
  Back
</a>
```

Step 12: Delete (Recommended)

Controller mein GET Delete ke bajay ye better hai:

```
C#
public IActionResult Delete(int id)
{
    var emp = _context.Employees.Find(id);

    if(emp != null)
    {
        _context.Employees.Remove(emp);
        _context.SaveChanges();
    }

    return RedirectToAction("Index");
}
```

Final Project Structure

```
EmployeeApp
├── Controllers
│   └── EmployeeController.cs
├── Models
│   └── Employee.cs
├── Data
│   └── ApplicationDbContext.cs
├── Views
│   └── Employee
│       ├── Index.cshtml
│       ├── Create.cshtml
│       ├── Edit.cshtml
│       └── Details.cshtml
├── appsettings.json
├── Program.cs
├── Migrations
│   ├── InitialCreate.cs
│   └── ApplicationDbContextModelSnapshot.cs
└── EmployeeApp.csproj
```

Viva Questions (Bahut Important)

Q. DbSet kya hota hai?

A. Table represent karta hai.

```
C#
public DbSet<Employee> Employees { get; set; }
```

Q. Migration kya hai?

A. Model ke basis par database schema create/update karne ki process.

Q. Add-Migration kya karta hai?

A. Migration file generate karta hai.

Q. Update-Database kya karta hai?

A. Database aur tables physically create karta hai.

Q. DbContext kya hai?

A. Application aur Database ke beech bridge hai.

```
C#
public class ApplicationDbContext : DbContext
```

Q. EF Core ke CRUD methods?

```
C#
Add()
Find()
Update()
Remove()
SaveChanges()
```

Ye pura Employee CRUD project exam ke liye sufficient hai. Iske baad hum **MVC Application with Validation, Student Registration Form**, aur **ViewBag/ViewData practicals** bhi cover kar sakte hain, jo CDAC practicals mein frequently aate hain.

ab mai program run kar sakta hu

Haan, **agar tumne ye sab steps complete kar liye hain**, to project run kar sakte ho:

- ✓ EF Core packages install kiye
- ✓ Employee.cs banaya
- ✓ ApplicationDbContext banaya
- ✓ appsettings.json mein connection string add ki
- ✓ Program.cs configure kiya
- ✓ EmployeeController banaya
- ✓ Views (Index/Create/Edit/Details) banayi
- ✓ Migration run ki:

```
PowerShell
Add-Migration InitialCreate
Update-Database
```

Run karne se pehle ek baar check kar lo

1. Build Project

Visual Studio:

Build → Build Solution

Shortcut:

```
Ctrl + Shift + B
```

Agar **0 Errors** aa rahe hain to good.

2. Check LocalDB

SQL Server Object Explorer mein dekh sakte ho:

```
(localdb)\MSSQLLocalDB
├── EmployeeDB
│   └── Tables
│       └── Employees
```

Agar table bani hai to migration successful hai.

3. Run Project

```
F5
```

ya

```
Ctrl + F5
```

Browser open hoga:

```
https://localhost:xxxx
```

Aur route ke hisab se:

```
https://localhost:xxxx/Employee
```

ya

```
https://localhost:xxxx/Employee/Index
```

Agar Error Aaye To

Mujhe ye 3 cheezein bhej dena:

1. **Program.cs**
2. **appsettings.json**
3. **Exact error screenshot/text**

Main turant bata dunga kis line mein problem hai.

Expected Result

- Employee List page open hoga.

- "Add Employee" pe click karoge.
- Employee save hoga SQL Server LocalDB mein.
- Index page par record show hoga.
- Edit/Delete/Details kaam karenge.

Agar abhi run kar rahe ho, aur koi error aaye, to screenshot bhej do.